

B05 - Cloud Platform Essentials

Course 2B DA-to-DE Bridge 2026 Fast Track | Beginner-first |

Bridge purpose

Learn enough cloud/platform reasoning to enter Course 3 without turning this Bridge into a Fabric administration course. No paid/live tenant is required.

Study loop

Lesson -> optional visual/reference -> follow once -> genuine workbook attempt -> save evidence -> protected review -> close review -> changed retry -> validate -> teach back -> mastery.

Scope boundary

B05 does not teach provisioning, Spark, notebooks, pipelines, deployment, secrets implementation or Fabric administration. C3-00 and C3-07 own those skills. Here you learn platform boundaries, access, freshness, monitoring and handoff contracts.

This book is your teacher

Use one lesson at a time. Every lesson keeps the same beginner-first sequence even when the engineering idea becomes more technical. Do not open protected review before you save a genuine attempt. After review, close it and complete the changed retry before mastery.

Daily loop understand -> picture -> new words -> watch/read if useful -> follow once -> fresh task -> validate -> teach back -> protected review -> changed retry -> evidence

Contents / Index

Lesson	Title	Pages
B05-L01	Cloud data mental model: storage, compute, network and identity	3-4
B05-L02	Fabric, OneLake, lakehouse and warehouse as one concrete example	5-6
B05-L03	Permissions, governance and private-source connection boundaries	7-8
B05-L04	Freshness, monitoring and failure ownership	9-10
B05-L05	Analyst-to-engineer platform collaboration and handoff	11-12

Resource and Gate rule

Official/current references control product-specific claims. Videos are visual companions with an exact action afterwards; the book + local case remains sufficient. After all five lessons and HomeFix Mini-Lab: attempt Gate A independently. Weak A -> targeted repair only. Gate B/C remain sealed until explicit full-retest assignment.

Protected assessment rule

Gate B/C and protected review material remain sealed until the correct saved-attempt -> review/repair -> explicit reassignment route allows them.

B05-L01 - Cloud data mental model: storage, compute, network and identity

Why it matters

Cloud diagrams feel like a list of product names until you separate four questions: where data lives, what work runs on it, how systems can reach each other, and who/what is allowed to act. Engineers make those boundaries explicit before selecting services.

What you are learning

Classify a data requirement into storage, compute, network/connection and identity; explain why a service name alone is not an architecture decision.

Prerequisite

Course 1 report refresh, sharing and basic cloud awareness. You do not need Azure/Fabric administration.

Picture it in your head

Mental picture

Picture a warehouse building. Storage is the shelves, compute is the workers/machines, network is the roads and locked doors, identity is the badge that says who may enter and what they may do.

New words before use

storage	where data is retained
compute	the processing resources that read/transform/query data
network boundary	the path and restrictions between systems
identity	a human, group or application identity used to authorize actions
service principal	a non-human application identity used for automated access

Watch / visual source

Use Microsoft Fabric overview as current reference. Optional visual: John Savill 02:49-09:44. After watching, close it and draw four boxes: storage / compute / network / identity; place one Distribution source decision in each.

[Microsoft Fabric overview](#)

[John Savill - Microsoft Fabric Overview](#)

Analyst -> engineer transition

Bridge transition

What you already know as an analyst: a report has a source, refresh and access.

What changes in engineering: each dependency becomes an explicit architecture boundary.

Why engineers care: failures and security problems often happen at a different boundary from the visible report.

Small worked example: WMS_SQL is on-prem storage; a cloud engine may compute over landed data; a secure private connection crosses the network boundary; SPN-PIPE can be the scheduled identity.

Real engineering version: C3-07 will provision and operate concrete platform resources. Here you learn the decision grammar first.

B05-L01 - Follow, prove and transfer

Follow with me once

Open B05_CLOUD_PLATFORM_PRACTICE_RC1.xlsx -> Source_Inventory and L01_Cloud_Mental_Model. Classify WMS_SQL, Carrier_API, Vendor_Files and Semantic_Model across the four dimensions.

What you should see

You should see that one source can create several different requirements. "Use Fabric" is not yet an answer because it does not state data placement, processing, connection path or identity.

Why it worked

The method works because requirements constrain architecture. A private SQL source creates a connection problem even if the destination service is healthy; a scheduled pipeline creates an identity problem even if a human can access the workspace.

Common beginner mistakes

- Choosing a product before writing the requirement.
- Treating storage and compute as the same thing.
- Assuming every source needs a gateway.
- Using a person's login as an automation identity without discussing ownership.

If stuck

Recovery ladder

Take one source only. Ask four literal questions: Where does its data live? What must process it? Can the destination reach it? Who/what should authenticate?

Small version alone

Fresh case: an e-commerce company has a cloud API, a private finance database and a daily file drop. Write one storage, compute, network and identity concern for each.

Validate

Every decision points to a stated source/freshness/security requirement. Cloud sources are not automatically labeled "gateway"; private sources are not magically reachable.

Teach it back

Without notes, explain the four-box model in 60-90 seconds and give one failure example for each box.

Protected review + changed retry

Save your first attempt before review. After review, change WMS freshness from 60 minutes to 10 minutes and decide which boundaries might need a different design.

Evidence / mastery + next route

Finished when

Save four-box sketch + source classification + one failure chain + changed retry + confidence 1-5. Ready only when you can classify a fresh case without a service-name prompt.

B05-L02 - Fabric, OneLake, lakehouse and warehouse as one concrete example

Why it matters

You need one real platform to make cloud ideas concrete, but memorizing Fabric screens would waste Bridge time. The useful skill is mapping platform pieces to requirements and knowing when storage, item type and consumer experience differ.

What you are learning

Explain tenant/workspace/item/OneLake at a beginner level; compare lakehouse and warehouse fit; reason about copy versus reference/shortcut without requiring a live tenant.

Prerequisite

B05-L01 four-box model plus Course 1 Power BI workspace familiarity.

Picture it in your head

Mental picture

Think of one campus (tenant), several secured rooms (workspaces), tools/objects inside each room (items), and a common organization-wide storage foundation underneath (OneLake). A lakehouse and warehouse are different working shapes over data, not moral "better/worse" choices.

New words before use

tenant	top-level organizational Fabric boundary
workspace	collaboration/security container for Fabric items
item	a Fabric object such as lakehouse or warehouse
OneLake	Fabric's unified logical data lake
lakehouse	data-lake storage with table/query capabilities for engineering/analytics
warehouse	relational analytical store optimized for SQL-centric warehouse work
shortcut/reference	access data in place rather than copying it when supported and appropriate

Watch / visual source

Read current OneLake and Fabric storage guidance. Optional visuals: Guy in a Cube 01:20-05:50; John Savill 43:28-50:18 and 1:00:43-1:13:12. After watching, write one lakehouse-fit case, one warehouse-fit case and one case where avoiding a second copy matters.

[What is OneLake?](#)

[Data storage options in Fabric](#)

[Guy in a Cube visual](#)

[John Savill Fabric overview](#)

Analyst -> engineer transition

Bridge transition

What you already know as an analyst: workspaces and semantic models organize reporting work.

What changes in engineering: you reason about shared storage, multiple data workloads and how data is exposed without unnecessary copies.

Why engineers care: duplication, ownership and workload choice affect reliability, cost and governance.

Small worked example: retain large raw files in shared storage; use a lakehouse-style path for file/table engineering and a warehouse when SQL-relational serving is the dominant need; do not copy a huge archive merely because another engine needs access if a governed reference is suitable.

Real engineering version: C3-07 implements these choices. B05 only requires an evidence-backed architecture decision.

B05-L02 - Follow, prove and transfer

Follow with me once

Open L02_Fabric_OneLake. Draw tenant -> workspace -> item -> OneLake/data. For four Distribution needs, choose lakehouse / warehouse / reference-copy pattern and write the rejected alternative.

What you should see

You should see decisions tied to workload and data behavior, not “lakehouse is modern so choose lakehouse.” OneLake does not mean every user can read every item/data set.

Why it worked

Separating organization/workspace/item from the storage foundation prevents a common beginner error: confusing shared storage architecture with unrestricted access or one universal compute engine.

Common beginner mistakes

- Treating Fabric as the definition of cloud data engineering.
- Saying OneLake means one physical copy in every case or universal read permission.
- Choosing lakehouse/warehouse by popularity instead of workload.
- Copying large data with no reason or ownership plan.

If stuck

Recovery ladder

Return to two questions: What shape/use must the consumer have? Does the data need to be copied, or can an authorized engine reference it safely?

Small version alone

Fresh case: product telemetry files, finance SQL reporting and a 10-TB image archive. Choose a plausible storage/serving pattern for each and defend one rejection.

Validate

Every component solves a named requirement; access remains deliberate; copy/reference reasoning names freshness, ownership and duplication implications.

Teach it back

Explain tenant, workspace, item and OneLake to a non-technical manager without using circular definitions.

Protected review + changed retry

After saving attempt and using review, change the large archive requirement from “monthly analysis” to “multiple engines need daily access.” Re-evaluate copy/reference and serving choice.

Evidence / mastery + next route

Finished when

Save hierarchy sketch + choice matrix + rejected alternative + changed retry + own-words explanation. Ready when a new workload changes your choice for a reason.

B05-L03 - Permissions, governance and private-source connection boundaries

Why it matters

A report can be shared correctly while the platform is still over-permissioned, or a workspace can be configured correctly while a private source is unreachable. Engineers separate management permission, data permission, identity and network connectivity.

What you are learning

Use least privilege for humans/groups/services, distinguish workspace/item/data access, and identify when a private/on-prem source needs a secure connection path.

Prerequisite

B05-L01 and L02. Course 1 RLS/sharing is useful prior context, but this lesson does not assume cloud administration.

Picture it in your head

Mental picture

Imagine three locks: building-management lock, room/item lock, and data-cabinet lock. A badge that opens one lock does not automatically open the others. A private source is in another building, so you also need a permitted road/bridge.

New words before use

least privilege	grant only the access needed for the required job
control plane	permissions to create/manage platform objects
data plane	permissions to read/write the data itself
group identity	permissions assigned to a managed set of people
gateway/private connection	controlled path that lets cloud services reach a network-restricted source

Watch / visual source

Use current Microsoft workspace roles, OneLake data access and on-prem data-source guidance. Optional John Savill 18:20-24:03 for workspace visual. After watching, separate “can manage the item” from “can read the data” for DE, analyst and executive personas.

[Fabric workspace roles](#)

[OneLake security/data access](#)

[Fabric on-premises source access](#)

[John Savill workspace visual](#)

Analyst -> engineer transition

Bridge transition

What you already know as an analyst: report sharing and RLS control what consumers see.
What changes in engineering: platform management, data access, service identity and source connectivity must all be designed independently.
Why engineers care: broad permissions and personal credentials create operational/security failures; private networks can block an otherwise healthy cloud pipeline.
Small worked example: GRP-DE may build items, GRP-EXEC may consume curated outputs, SPN-PIPE runs automation, and WMS_SQL uses a private connection path rather than a public credential workaround.
Real engineering version: C3-00/C3-07 handle secrets/configuration and actual platform setup; B05 proves you understand the boundaries before touching them.

B05-L03 - Follow, prove and transfer

Follow with me once

Open L03_Access_Connections. Map GRP-DE, GRP-Ops, GRP-Fin, GRP-Exec and SPN-PIPE to minimum duties. Then classify which sources require private connectivity.

What you should see

Human/group/service identities have different responsibilities. Cloud APIs do not receive an on-prem gateway just because gateways exist. Finance data does not inherit broad access simply because the item lives in a shared platform.

Why it worked

Least privilege reduces blast radius while explicit connection boundaries make failures diagnosable. The network can fail independently from identity, and identity can fail independently from the data itself.

Common beginner mistakes

- Giving Contributor because Viewer feels inconvenient.
- Using an analyst's personal token for production-like automation.
- Confusing workspace role with row/data authorization.
- Inventing a gateway for every internet/cloud API.

If stuck

Recovery ladder

For each persona/source write four columns: manage platform? read/write which data? authenticate as what? reach source through what path?

Small version alone

Fresh case: payroll DB on a private network, CRM SaaS API, external agency contractor. Design minimum identities/access and state which connection needs a private bridge.

Validate

Each access grant has purpose, scope and owner/review; private connection need follows source location; no credential is stored in learner evidence.

Teach it back

Explain why "Admin access" and "data access" are not the same thing, then give one offboarding risk of personal credentials.

Protected review + changed retry

After saved attempt + review, a vendor asks for Contributor access. Design a narrower alternative and explain what evidence would justify escalation.

Evidence / mastery + next route

Finished when

Save access matrix + connection map + one rejected broad permission + changed retry. Ready when you can reason from persona/source without memorizing role names.

B05-L04 - Freshness, monitoring and failure ownership

Why it matters

A green run is not the same as fresh, complete or trustworthy data. Engineers need evidence that data met the business freshness promise and a named response when it did not.

What you are learning

Turn a business freshness need into SLA/trigger/tolerance/owner/validation; diagnose stale-data incidents and require post-recovery proof.

Prerequisite

B05-L01-L03 and Course 1 refresh validation. No orchestration platform is required.

Picture it in your head

Mental picture

Imagine a train board. "Train engine running" does not prove the passenger arrived on time. You need scheduled arrival, actual arrival, delay tolerance, owner and a rule for informing waiting passengers.

New words before use

freshness SLA	agreed maximum age/arrival target for usable data
run status	whether a job execution reported success/failure
data freshness	how current the accepted data actually is
monitoring signal	evidence used to detect a condition
incident owner	person/team accountable for diagnosis/recovery/communication
post-recovery validation	checks proving recovered output is complete/correct/fresh

Watch / visual source

Use current Fabric monitoring hub for run/history/error concepts and Microsoft on-premises gateway overview for private-source dependency. After reading, write one signal -> owner -> recovery -> validation -> consumer message chain for each Distribution incident.

[Fabric monitoring hub](#)

[On-premises data gateway](#)

Analyst -> engineer transition

Bridge transition

What you already know as an analyst: refresh and control totals tell you whether a report is current/trustworthy.

What changes in engineering: freshness becomes an operational contract with monitoring, owner, recovery and downstream communication.

Why engineers care: a pipeline may be green while source data is late, partial or stale.

Small worked example: Carrier_API target 30 min; if OAuth expires, signal is 401, owner repairs credential path, rerun is not enough - count/window/freshness must be revalidated, then consumers get status.

Real engineering version: C3-03/C3-08 implement orchestration/observability. B05 teaches the acceptance logic first.

B05-L04 - Follow, prove and transfer

Follow with me once

Open L04_Freshness_Monitoring. Define SLA, schedule/trigger, tolerance, owner and validation for at least three flows. Then classify the three provided incidents.

What you should see

You should see separate technical and data states. A successful rerun still requires source-to-output completeness and freshness checks.

Why it worked

Operational contracts make “is it ready?” answerable. Without a target and validation signal, monitoring becomes decorative; without an owner, a detected failure can remain unresolved.

Common beginner mistakes

- Equating “job succeeded” with “data is good.”
- Rerunning until green with no reconciliation.
- No owner for late external files.
- Promising “real time” without a measurable threshold.

If stuck

Recovery ladder

Write the sentence: “Consumers may use this output when ____.” Fill it with freshness, completeness and quality evidence. Then ask who acts when any clause fails.

Small version alone

Fresh case: daily supplier file is 8 hours late but yesterday’s dashboard refresh is green. Decide whether output is acceptable and write the communication/recovery path.

Validate

For every critical flow: freshness target + detection + owner + recovery + post-recovery validation + consumer status are explicit.

Teach it back

Explain in 60-90 seconds why green run status can coexist with stale data.

Protected review + changed retry

After review, change a flow from hourly to 15 minutes and identify which assumptions/owners/cost/reliability questions must be revisited before accepting it.

Evidence / mastery + next route

Finished when

Save freshness contract + incident matrix + post-recovery validation + changed retry. Ready when you can reject a green-but-stale output with evidence.

B05-L05 - Analyst-to-engineer platform collaboration and handoff

Why it matters

The Bridge succeeds only when your analyst knowledge becomes usable engineering input. Vague requests such as “make it real time and secure” force engineers to guess and make later validation impossible.

What you are learning

Write a platform handoff with source/grain/use, freshness, access, owner, failure expectations, acceptance evidence, change impact and known limitations.

Prerequisite

B05-L01-L04 plus Course 1 business analysis/handoff discipline.

Picture it in your head

Mental picture

Think of a relay race. The analyst does not throw a vague idea over the wall; the baton has labels: what data, for which decision, when it must arrive, who may use it, what proves correctness and what happens on failure.

New words before use

acceptance criterion	objective condition that determines whether delivery is acceptable
handoff contract	shared written expectations between requestor/builder/operator
lineage	record of upstream/downstream data dependencies
impact analysis	checking what downstream work may change when an upstream source/definition changes
limitation	known boundary that must not be hidden

Watch / visual source

Use current Microsoft Fabric overview for shared workload context and Fabric lineage guidance for dependency/impact thinking. After reading, add upstream/downstream impact and owner to your handoff and state what must be revalidated after a source change.

[Microsoft Fabric overview](#)

[Fabric lineage](#)

Analyst -> engineer transition

Bridge transition

What you already know as an analyst: a good analysis brief defines decision, metric, data and validation.

What changes in engineering: the handoff also defines system boundaries, freshness, access, failure/recovery ownership and reproducible acceptance evidence.

Why engineers care: untestable requirements become unstable pipelines and argument-driven handoffs.

Small worked example: Inventory Availability requires named sources/keys, hourly maximum age, GRP-Ops consumer access, DE pipeline owner, completeness/control checks and a failure message if the SLA is missed.

Real engineering version: C3 uses these contracts to build/run actual systems; B05 proves you can specify and defend them.

B05-L05 - Follow, prove and transfer

Follow with me once

Open L05_Collaboration_Handoff. Write one Distribution output contract and explicitly split analyst, data-engineering and platform/security responsibilities.

What you should see

The receiver can objectively say accepted/not accepted. The contract avoids implementation micromanagement but leaves no ambiguity about requirements and evidence.

Why it worked

A contract lets engineering choices evolve without losing the business promise. Lineage/impact notes prevent an upstream change from silently invalidating downstream consumers.

Common beginner mistakes

- Writing “real time” instead of a threshold.
- Writing “secure” instead of personas/scopes/owners.
- Specifying a favorite service with no requirement.
- No acceptance evidence or failure communication.
- Hiding assumptions/limitations.

If stuck

Recovery ladder

Use six prompts: source/use; grain/definition; freshness; access; failure owner; acceptance proof. Then add one limitation and one downstream impact.

Small version alone

Fresh case: customer churn score for sales needs hourly refresh, confidential CRM data and one external contractor. Write the handoff without choosing every implementation detail.

Validate

A second person can build a plausible solution, test acceptance and identify who owns a failure without asking what the vague words meant.

Teach it back

Defend your handoff in 60-90 seconds: requirement, risky assumption, validation, failure route and one rejected alternative.

Protected review + changed retry

After review, stakeholder changes daily freshness to hourly. Write the impact questions and update only the clauses that actually change.

Evidence / mastery + next route

Finished when

Save final handoff + responsibility split + lineage/impact note + rejected alternative + changed retry. Ready for Mini-Lab when all five lessons have genuine saved attempts, protected review, fresh retry and own-words evidence.